Automation Is Not Intent:
Measuring and Fixing Fraud-Model Failures on Legitimate AI Shopping Agents

Rishabh Pande
Independent researcher
rish.pande@gmail.com

October 2026

Abstract

Fraud models learned that "bot-like" means "risky": no mouse movement, instant form fills, headless browsers, datacenter IPs. AI shopping agents now make legitimate purchases with exactly these signals. We call this the *bot–fraud conflation* and measure it. We introduce AGENTRISK, an open semi-synthetic benchmark of nine session classes, including four agent-era attacks: fraud via unverified agents, *laundering* through lax agent platforms, prompt-injection *hijacking* of a user's own agent, and signature spoofing. Agent telemetry is grounded in real measurements of the automation primitives agent frameworks use. At a 2% human decline rate, a standard gradient-boosted fraud model declines 94.6% of legitimate agent purchases. The obvious fixes each fail differently. Retraining on agent traffic creates a *safe harbor*: catch rate on fraud via agents falls from 98.4% to 53.6%. Allowlisting verified agents stops none of the laundering or hijacks. Agent-identity features make hijacks harder to catch (0.2%). Labeling agent-era fraud brings the false declines back. Identity standards say *who* acts, not *for whom* or *within what scope*. We propose AGENTGATE, a verify–scope–score policy combining agent identity, a delegation chain with credential binding, and mandate-consistency checks, with step-up instead of decline for unattested automation. It cuts false declines on verified agents to 1.7% and stops 92.7% of laundering and 70.2% of hijacks, none seen in training, with human-traffic performance unchanged. Sessions from a real production agent confirm the pattern: the legacy model declines 88.5% of them when cloud-hosted, while AgentGate declines 1.9% once they are verified. We release the benchmark, telemetry harness and evaluation protocol.

1 Introduction

In less than two years, every layer of the payment stack announced support for AI agents that shop on a user's behalf. Card networks shipped agent-recognition programs (Visa's Trusted Agent Protocol [24], Mastercard Agent Pay [17]). Platforms published checkout protocols for agents (the Agentic Commerce Protocol from Stripe and OpenAI [21], Google's Agent Payments Protocol, AP2 [11]). The IETF adopted a working draft for cryptographically signed bot traffic [18]. Yet the fraud models that decide whether each of these purchases goes through were trained on a world in which automation at checkout was almost always an attack. Industry has started to notice the result: legitimate agents are being declined as bots [22]. To our knowledge, however, there is no rigorous measurement of how large the effect is, why it happens, or whether the proposed fixes work.

This paper argues that the problem is not detection. Recent measurement studies show that AI web agents are easy to fingerprint, with up to 97% accuracy in separating agent architectures from humans and

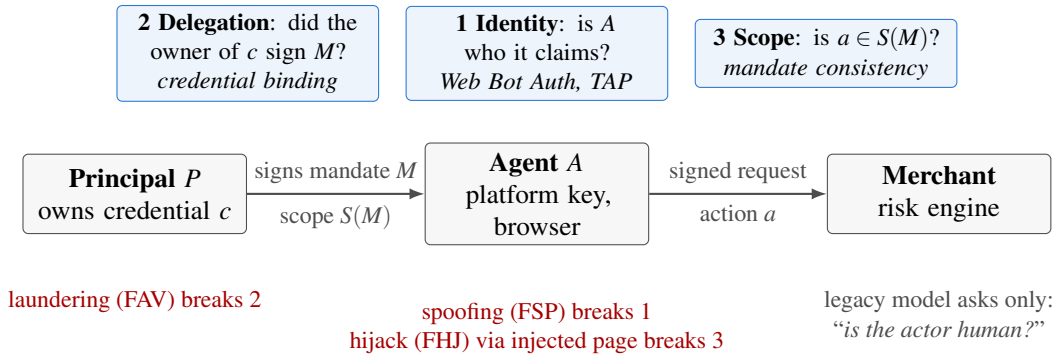


Figure 1: An agent purchase is authorized only if three independent checks pass (Equation (1)). Behavioral bot signals answer none of them; agent-identity standards answer only the first. Each agent-era attack in AGENTRISK breaks a different check.

crawlers [9, 15, 25]. The problem is that fraud models use "is this automated?" as a stand-in for "is this authorized?" Before agents, the two questions had nearly the same answer. Now they do not. We call this the **bot–fraud conflation**, and we show that the obvious ways of removing it each create a new attack surface.

Three questions, not one. A purchase by an agent (Figure 1) is legitimate when three things hold. The agent is who it claims to be (*identity*). It acts for a principal who controls the payment credential (*delegation*). And the specific action falls within what that principal authorized (*scope*). Behavioral bot signals answer none of these. Signed-agent standards such as Web Bot Auth answer only the first, and the IETF draft explicitly leaves authorization and delegation out of scope [18]. This is a classic *confused-deputy* setting [13]. An authenticated agent can be steered, by a fraudster who controls it or by a prompt injection [12], into spending authority that belongs to someone else.

Contributions.

1. **Measurement of the conflation.** We give the first quantitative estimate of false declines on legitimate agent traffic. We ground it in real telemetry from the six automation primitives that agent frameworks use (Section 4.1). In the benchmark, a conventional fraud model declines legitimate agents 47× more often than humans.
2. **AGENTRISK, an open benchmark.** It covers nine session classes, including four agent-era attacks that existing fraud datasets lack: fraud via unverified agents, laundering through permissive agent platforms, prompt-injection hijacking, and signature spoofing. It comes with an evaluation protocol that fixes the human decline rate and reports agent false declines and per-attack catch rates (Section 4).
3. **Three failure modes of naive fixes.** Retraining produces a *safe harbor*, allowlisting produces a *laundering channel*, and identity features produce a *verification paradox* in which hijacks get through more easily (Section 6.2).
4. **AGENTGATE.** A verify–scope–score policy with delegation-chain and mandate-consistency features and step-up routing. It keeps false declines on verified agents near the human rate while stopping most agent-era attacks, including ones never seen in training (Sections 5 and 6).
5. **A check against a real production agent** (Claude computer use, Section 6.7). It confirms the conflation and the fix, and corrects one simulator assumption: real agents need not expose automation flags.
6. **Design guidance** for protocol designers, issuers and merchants (Section 7).

2 Background

The agentic commerce stack. Four kinds of protocol are emerging. (i) *Agent identity*: Web Bot Auth signs each HTTP request with RFC 9421 message signatures [3], using keys published in a directory [18]. Visa's Trusted Agent Protocol and Mastercard Agent Pay build on this so that merchants can recognize registered agents and tell browsing from paying [4, 17, 24]. (ii) *Delegation*: AP2 represents user authorization as signed *mandates*, encoded as verifiable credentials. An *intent mandate* carries constraints such as price caps, categories and timing, and a *cart mandate* authorizes a specific basket [11]. (iii) *Scoped credentials*: ACP's Shared Payment Token is limited to one merchant and one cart total [21], and network "agentic tokens" bind a card to an agent and a consumer [17]. (iv) *Checkout APIs* that let agents bypass the web UI altogether. Adoption is partial and uneven. During the transition, merchants see a mix of humans, verified agents, unverified agents (for example, open-source browser agents a user runs on their own machine), and attackers who can imitate any of these.

What fraud models look at. Production card-not-present fraud models are usually gradient-boosted trees over three kinds of feature [5, 6]. *Transaction and account* features cover amount, account age, address mismatches and velocity. *Device and network* features cover IP reputation, datacenter ranges and browser fingerprints [23]. *Behavioral biometrics* cover mouse dynamics, keystroke timing, field-fill patterns and event trust [2, 14]. The behavioral and device groups were designed to catch bots, and a label of "bot at checkout" has historically been a strong fraud indicator. That is the prior that agents now violate.

3 Problem Formulation and Threat Model

A checkout session x is executed by an *actor* A (a human or an agent) using payment credential c. For an agent session, there is a *principal* P on whose behalf A claims to act, a possibly empty *mandate* M signed by P with scope $S(M)$ (merchant, category, amount cap, delivery address), and the concrete action a (cart, amount, ship-to). We call the session *authorized* iff

$$\underbrace{\mathsf{Auth}(A)}_{\text{identity}} \wedge \underbrace{\mathsf{Owns}(P,c) \wedge \mathsf{Signed}_P(M)}_{\text{delegation}} \wedge \underbrace{a \in S(M)}_{\text{scope}}. \tag{1}$$

For a human session, (1) collapses to the familiar question of whether the person at the keyboard owns c. Behavioral features were useful because, for human sessions, "a human is present" was strong evidence of that. For agent sessions, behavior describes the software, not the principal, and carries no information about any conjunct of (1).

Adversary. We consider fraudsters who hold stolen credentials or hijack accounts, and who can run any agent framework, sign up to agent platforms, and plant content on web pages that agents read. They cannot forge the signatures of honest agent platforms or principals. Table 1 lists the attack classes. Classic fraud (FH, FB) is present in historical training data. The agent-era classes (FAU, FAV, FHJ, FSP) are not, which matches how a deployed model would first meet them.

4 The AGENTRISK Benchmark

4.1 Grounding: what automation primitives actually emit

LLM agents act on pages through a small set of primitives exposed by browser-automation libraries: DOM-level fills, CDP text insertion, synthesized keystrokes, or coordinate-level mouse and keyboard control (as in

Table 1: Session classes in AGENTRISK. "Era" indicates whether a class appears in pre-agent training data.

Class	Era	Description
H	–	Legitimate human; desktop or mobile (no mouse events), some with browser autofill.
AU	–	Legitimate *unverified* agent (e.g., an open-source browser agent run by the user); no signature.
AV	–	Legitimate *verified* agent: valid signature, registered platform, mandate signed by the principal.
FH	1	Manual fraud with stolen credentials or a taken-over account.
FB	1	Scripted bot: card testing or automated account takeover.
FAU	2	Fraudster drives an unverified agent with stolen credentials.
FAV	2	*Laundering*: fraudster gets a *valid* agent identity from a permissive platform and signs a mandate as a principal they control; the stolen card is not bound to that principal.
FHJ	2	*Hijack*: a legitimate user's verified agent is redirected by indirect prompt injection; identity and principal are genuine, the action is not.
FSP	2	Spoofed signature that fails verification.

Table 2: Measured client-side telemetry from browser-automation primitives on a five-field checkout (means over 10 runs; time is the median). Webdriver and headless-UA flags are shown as default/stealth.

Primitive	mouse-moves	keydown /input	untrusted share	IKI (ms)	submit (s)	webdriver flag	headless UA
DOM `fill`	1	0/5	0.00	–	0.11	1/0	1/1
JS value + `dispatchEvent`	0	0/5	0.55	–	0.01	1/0	1/1
CDP `insertText`	6	0/5	0.00	–	0.24	1/0	1/1
keystrokes, no delay	1	76/76	0.00	1.6	0.20	1/0	1/1
keystrokes, 100 ms delay	1	76/76	0.00	103.2	7.91	1/0	1/1
computer-use style	157	76/76	0.00	107.5	10.00	1/0	1/1

computer-use agents). We built an instrumented checkout page whose collector records what a client-side risk SDK sees: event counts, the share of untrusted events, keydown-to-input ratio, inter-key intervals, mouse-path straightness, time to submit, `navigator.webdriver`, and the user-agent string. We completed the same five-field checkout 10 times with each primitive in headless Chromium driven by Playwright, with and without the common `AutomationControlled` stealth flag (Table 2).

Three observations shape the benchmark. First, the most common agent primitives (`fill`, `insertText`) produce *input events with no keydowns*. Browser autofill does the same, which is why legacy models cannot simply hard-block on it. Second, synthesized keystrokes arrive about 1.6 ms apart, two orders of magnitude faster than human typing, unless a delay is configured. Computer-use-style control is the most human-like primitive (mean inter-key interval 107 ms, with real mouse paths). Third, the widely used stealth flag hides `navigator.webdriver` but leaves "HeadlessChrome" in the user-agent string. Evasion is partial and leaves its own traces, consistent with prior findings that stealth measures can increase detectability [9].

4.2 Session generator

Real labeled agent-commerce data does not yet exist publicly, and production fraud data cannot be released. We therefore follow the semi-synthetic design common in fraud research [6]. Agent and bot behavioral

telemetry is sampled around the measured per-primitive centers in Table 2, with frameworks mixing primitives (DOM fill 40%, CDP insertion 20%, keystrokes 10%, computer-use 30%). A configurable share of agents are *humanized*, with jittered typing, curved mouse paths and a patched webdriver flag. Human, transaction, network and delegation features are parametric. Every assumption is listed in Appendix A and exposed as a parameter, and Section 6.5 sweeps the ones that matter most. We deliberately make classic fraud hard to separate on transaction features alone (legacy recall on manual fraud is about 71.4% at our operating point), so that behavioral signals carry real weight, as they do in practice.

Features fall into four groups. **B** (behavioral, 10 features) and **D** (device/network, 4) are what current models use. **T** (transaction/account, 8) is standard. **A** (agent-aware, 14) is the new group and has three parts:

- *Identity*: signature present and valid; KYC tier of the agent platform from the key directory.
- *Delegation*: mandate present and valid; principal account age; whether the credential is *bound* to the principal (e.g., via a network agentic token); mandate age; chain depth; human present or not.
- *Consistency*: amount relative to the mandate cap; merchant in scope; category in scope; ship-to matches the principal.

Two realistic confounders are built in: agents retry more (higher attempt velocity), and cloud agents share egress IPs across many users (very high per-IP session counts).

4.3 Protocol and metrics

Models train on an era-1 world (150k human sessions, 1.5k manual fraud, 1k bot fraud), optionally augmented with 16k labeled legitimate-agent sessions. They are tested on 150k humans, 10k each of AU and AV, and 2k of each fraud class. We fix the operating point by calibration on held-out humans so that **2% of legitimate human sessions are declined**, and report per class:

- **Agent false-decline rate (AFDR)**: the share of legitimate agent sessions that end declined, including failed step-ups.
- **Catch rate**: the share of each fraud class that ends declined.
- **Step-up rate**: friction imposed on legitimate traffic.

Step-up outcomes are modeled explicitly. Legitimate principals complete 92% of step-ups. Hijack victims who are shown the redirected cart approve 10%, and fraudsters facing cardholder authentication pass 5%. All results are means over 5 seeds, with independent data at each seed.

5 Policies

All learned components are LightGBM classifiers [16] with identical hyperparameters (300 trees, 31 leaves). **Legacy** uses B+D+T trained on era-1 data, which is the status quo. **Retrained** uses the same features plus labeled legitimate agents. This is the "just add the new traffic" fix. **+Allowlist** approves any session with a valid agent signature, the "trust verified agents" fix suggested by agent-identity programs. **Identity-aware** adds the identity features to the retrained model.

AGENTGATE. Our policy separates what can be *verified* from what must be *learned* (Algorithm 1). Deterministic checks come first. Signatures and mandates that fail verification are declined. For verified agents, an unbound credential or an action outside the mandate's scope (with a 10% amount tolerance for taxes and shipping) is *stepped up to the principal or issuer* rather than approved or declined. Remaining sessions are scored by a model over B+D+T+A. Unattested automation in the grey band (scores between the 90th and 98th human percentile) is stepped up instead of approved. The design rests on three principles. (i) *Automation is not evidence*: being a bot neither raises nor lowers risk once identity is established. (ii) *Authorization is checkable*: scope violations are a matter of protocol semantics, not statistics, so they should

Algorithm 1 AGENTGATE decision for a session x

$s \leftarrow f_\theta(x_{B,D,T,A})$ ▷ learned score, threshold τ at 2% human declines
if signature present $\wedge$ invalid **or** mandate present $\wedge$ invalid **then return** DECLINE
end if
if $s \geq \tau$ **then return** DECLINE
end if
if signature valid **then**
    if credential not bound to principal **or** $a \notin S(M)$ **then return** STEP-UP
    end if
else if automation detected $\wedge$ $s \geq \tau_{\text{grey}}$ **then return** STEP-UP ▷ unattested automation
end if
return APPROVE

Table 3: Share of sessions declined (%) at 2% human decline rate, mean over 5 seeds. Across seeds the standard deviation is at most 1.7 points, except for five weak-baseline cells on FSP and FAV, where it is at most 4.7. For legitimate classes (H, AU, AV) lower is better; for fraud, higher is better. Shaded cells: more than 20% false declines on legitimate agents, or fewer than half of a fraud class stopped.

	Legitimate			Era-1 fraud		Agent-era fraud (unseen)			
Policy	H	AU	AV	FH	FB	FAU	FAV	FHJ	FSP
Legacy	2.0	94.5	94.6	71.4	100.0	98.4	98.4	96.1	98.5
Legacy + allowlist	2.0	94.5	0.0	71.4	100.0	98.4	0.0	0.0	98.5
Retrained	2.0	3.9	4.1	71.4	97.8	53.6	53.3	15.3	53.4
Retrained + allowlist	2.0	3.9	0.0	71.4	97.8	53.6	0.0	0.0	53.4
Identity-aware ML	2.0	6.4	0.0	71.3	98.5	59.7	18.4	0.2	28.3
AgentGate (ours)	2.0	7.2	1.7	71.2	99.4	71.4	92.7	70.2	100.0

not wait for labeled examples. (iii) *Ask, don't guess*: when evidence of delegation is missing, the cheapest signal is the principal's own confirmation.

6 Results

Table 3 gives the main results and Figure 2 summarizes the trade-off.

6.1 RQ1: How large is the conflation?

Finding 1. At a 2% human decline rate, the legacy model declines 94.6% of legitimate verified-agent purchases and 94.5% of unverified ones, about 47× the human rate. Its accuracy on the traffic it was built for is excellent (AUC 0.977 on humans vs. era-1 fraud), so the failure is invisible to standard monitoring.

Figure 3(a) shows why. The legacy model's score distribution for legitimate agents sits on top of the scripted-bot fraud distribution and far from humans. Only humanized agents partly escape (66.4% declined vs. 100.0% for DOM-fill agents). A merchant that monitors fraud-model AUC or human approval rates would see nothing wrong. The lost revenue appears only as abandoned agent sessions, which, unlike fraud losses, produce no chargeback signal [22].

6.2 RQ2: Why the obvious fixes fail

Figure 4 breaks results down by attack.

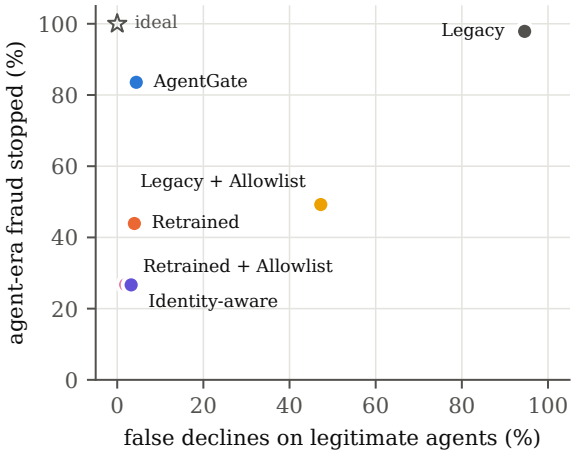


Figure 2: Failure-mode map. Each point is a policy at the same 2% human decline rate. The x-axis is the mean false-decline rate on legitimate agents (AU, AV); the y-axis is the mean catch rate over the four agent-era attacks. The legacy model sits on the right (blocks everything automated). The naive fixes move left but fall: they stop false declines by also stopping detection.

Finding 2 (safe harbor). Retraining on labeled legitimate agents cuts agent false declines to 3.9%/4.1% (AU/AV), but the agent fingerprint becomes a learned *signal of safety*. Catch rate on fraud through unverified agents falls from 98.4% to 53.6%, below the catch rate on manual fraud (71.4%). Behaving like a well-mannered agent is now a way to lower one's risk score.

The mechanism is simple. Once legitimate agents are labeled negative, splits on automation features route everything agent-like toward low scores. Era-1 fraud never looked like a legitimate agent, so nothing in training pushes back. This is concept drift [10] in its adversarial form: the attacker can move into the region the retraining just declared safe.

Finding 3 (laundering). Allowlisting verified agents stops 0.0% of laundering and 0.0% of hijacks. A valid signature proves which software sent the request, not who authorized the payment. Any agent platform with weak onboarding becomes a way to wash stolen credentials.

Finding 4 (verification paradox). Giving the model agent-*identity* features is *worse* than ignoring them for hijacks: 0.2% caught vs. 15.3% for the retrained model. Every verified session in training was legitimate, so the model learns "verified ⇒ safe." Hijacks carry genuine signatures from genuine platforms for genuine users.

6.3 RQ3: Does AGENTGATE fix it, and which parts matter?

Finding 5. AGENTGATE declines 1.7% of verified-agent and 7.2% of unverified-agent purchases at an unchanged human decline rate (2.0%). It stops 92.7% of laundering, 70.2% of hijacks, 100.0% of spoofs and 71.4% of fraud via unverified agents, close to the manual-fraud catch rate (71.2%). None of these attacks were in its training data.

The price of this is friction rather than declines. 20.8% of verified-agent sessions are stepped up (mostly those whose credential is not yet bound to the principal) and 10.2% of unverified-agent sessions. No human sessions are stepped up, because the grey-band routing applies only to automated traffic. Unverified legitimate

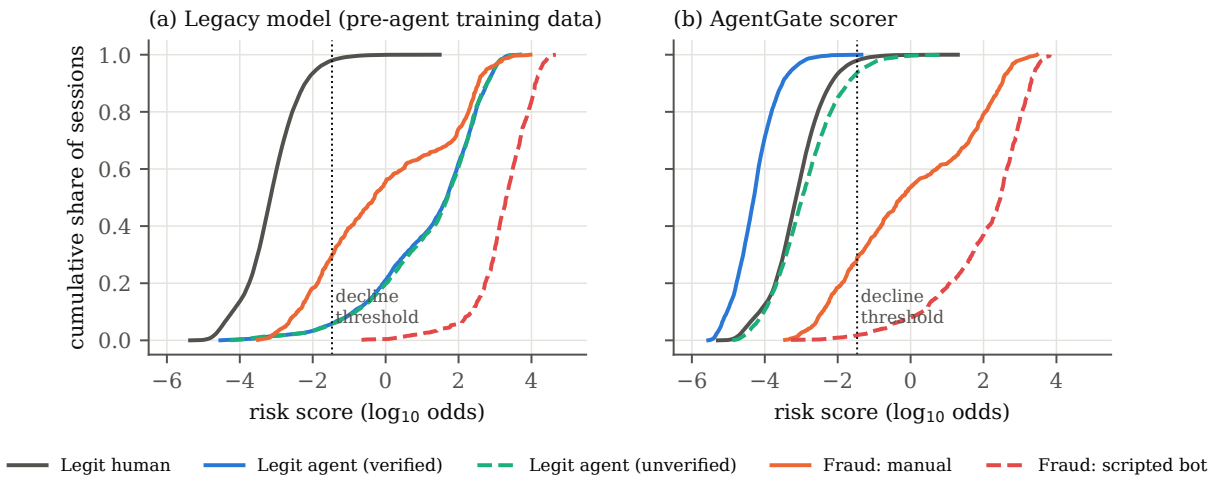


Figure 3: Cumulative score distributions (seed 0). (a) The legacy model scores legitimate agents (blue, aqua) like scripted bots (red), far above humans; almost all fall past the decline threshold. (b) Under AGENTGATE's scorer, legitimate agents move to the human side while classic fraud stays separable.

Table 4: Ablation of AGENTGATE (share declined, %, same protocol as Table 3).

Variant	H	AU	AV	FH	FB	FAU	FAV	FHJ	FSP
AgentGate (ours)	2.0	7.2	1.7	71.2	99.4	71.4	92.7	70.2	100.0
− all gates (ML only)	2.0	6.3	0.0	71.2	98.5	59.5	20.0	0.1	34.8
− scope (consistency) checks	2.0	7.3	1.7	71.4	99.4	71.7	92.4	17.8	100.0
− credential binding	2.0	7.0	0.1	71.3	99.4	71.1	28.4	65.9	100.0
− step-up for unattested automation	2.0	6.3	1.6	71.2	98.5	59.5	93.0	70.2	100.0
AgentGate, automation-invariant	2.0	6.9	1.7	62.3	95.7	70.6	93.6	70.1	100.0

agents still pay a higher price than verified ones, which gives agent developers a reason to adopt signing.

Ablation. Table 4 removes one component at a time. Without the scope checks, hijack catch falls from 70.2% to 17.8%. Without credential binding, laundering catch falls from 92.7% to 28.4%. With the gates removed and only the learned model kept, the system reproduces the verification paradox (FHJ 0.1%, FSP 34.8%). Every attack-specific improvement comes from a component that checks a conjunct of (1), not from more data. Removing the step-up for unattested automation lowers FAU catch to 59.5%. An *automation-invariant* variant that drops all bot telemetry lowers catch on manual fraud from 71.2% to 62.3% and on scripted bots from 99.4% to 95.7%, so behavioral signals remain useful for *unattested* traffic. The fix is to stop treating them as evidence against attested agents, not to discard them.

6.4 RQ4: Does labeled attack data close the gap?

A natural objection is that once agent-era fraud is labeled, any model will learn it. We add $k \in \{25, 100, 400\}$ labeled examples of each agent-era attack class to training (Figure 5). With 400 examples per class, the retrained model's catch rates rise to 91.9% (FAU), 91.9% (FAV) and 70.4% (FHJ). AGENTGATE already reaches 92.2% on laundering and 71.0% on hijacks with *zero* examples. But the labels have a second effect, which is the more important result:

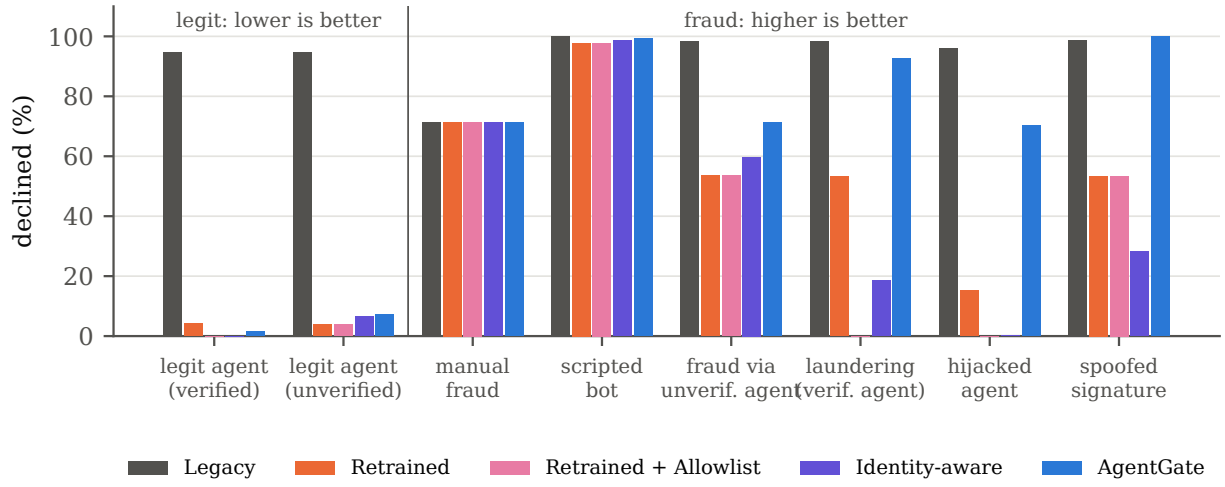


Figure 4: Share declined per session class for five policies. Each naive fix trades the false-decline problem for a specific blind spot: fraud via agents (Retrained), laundering and hijacks (Allowlist), or hijacks (Identity-aware).

Finding 6 (labels bring back the conflation). As labeled fraud-via-agent examples accumulate, models without delegation and scope evidence relearn "agent ⇒ risky." At 400 examples per class, the retrained model again declines 37.2% of legitimate agents, up from 4.1%. The identity-aware model declines 38.0% of verified agents. AGENTGATE's verified-agent false declines rise only to 10.0%, because its strongest evidence about agent sessions does not come from behavior.

When features cannot tell a legitimate agent from a fraudulent one, more labels only move the threshold between the two errors; they do not remove the trade-off. Breaking it requires features that separate the two classes, which are exactly the delegation and scope features. In production, each label is also a loss that has already happened. Verification gives protection from the first day.

6.5 RQ5: Sensitivity to simulator assumptions

We sweep the four assumptions most likely to differ in deployment (Figure 6): the share of humanized agents (0–80%), adoption of credential binding (50–95%), the share of cloud-hosted agents (20–100%), and the share of hijacks that stay within mandate scope (10–60%). Across all settings the legacy model's agent false-decline rate stays at or above 71.5%, and AGENTGATE's stays at or below 5.6%. Two sensitivities are worth noting. When binding adoption is low (50%), more verified agents are stepped up and AGENTGATE's AV false-decline rate rises to 3.9% (failed step-ups), which makes binding the main lever for issuers. When attackers keep hijacks inside the mandate scope, hijack catch falls from 82.0% to 54.7%. Tight mandate scopes are therefore a security property, not just a user-experience detail.

6.6 RQ6: What does it cost?

Following cost-sensitive evaluation practice [8], we convert outcomes into expected merchant loss per 10,000 checkouts. A false decline costs 30% margin on the order plus $10 of customer lifetime value. Approved fraud costs the order value plus a $20 chargeback fee. A step-up costs $0.25. Fraud makes up 1% of traffic, half of it agent-era. Table 5 varies the agent share of legitimate traffic. At 5% agent share, AGENTGATE lowers expected loss by 57% relative to the legacy model. At 20% the reduction is 84%, and at 40% it is 90%. Across the agent shares tested, the cheapest naive alternative (Retrained+Allowlist at 20%) costs 15,357

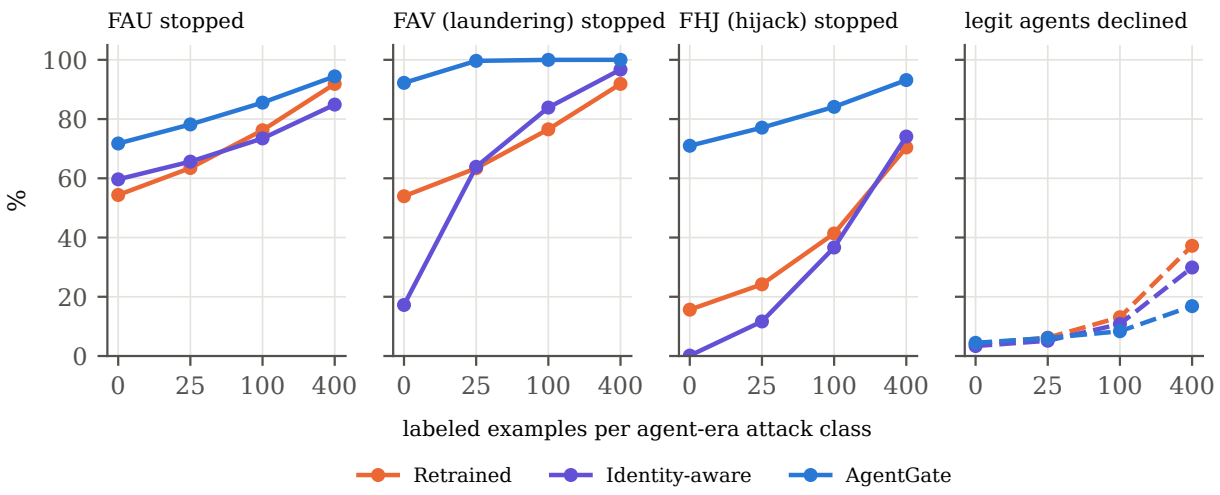


Figure 5: Adding labeled agent-era attacks to training (3 seeds). Left three panels: catch rates (higher is better). Right panel: false declines on legitimate agents (lower is better). Labels improve detection for every policy, but they push behavior-based models back toward declining legitimate agents.

Table 5: Expected loss ($) per 10,000 checkouts: false declines (FD), approved fraud (Fraud) and total including step-up friction, at three agent shares of legitimate traffic.

	5% agents			20% agents			40% agents		
Policy	FD	Fraud	Total	FD	Fraud	Total	FD	Fraud	Total
Legacy	24,283	1,719	26,002	75,131	1,719	76,850	142,927	1,719	144,647
Legacy + allowlist	15,619	6,004	21,623	40,474	6,004	46,477	73,613	6,004	79,617
Retrained	7,701	6,712	14,413	8,789	6,712	15,501	10,240	6,712	16,952
Retrained + allowlist	7,328	8,059	15,387	7,298	8,059	15,357	7,256	8,059	15,315
Identity-aware ML	7,526	7,712	15,238	8,208	7,712	15,920	9,117	7,712	16,829
AgentGate (ours)	7,743	3,479	11,241	9,077	3,479	12,633	10,856	3,479	14,488

against 12,633 for AGENTGATE. The constants are illustrative, so the ranking is the result to take away, not the dollar figures. Under any cost model that prices both false declines and fraud, legacy loss grows with agent adoption.

6.7 RQ7: Do the findings hold for a real production agent?

The benchmark's agent telemetry comes from automation primitives, not from a deployed agent. To check the central claim against a real one, we published the instrumented checkout from Section 4.1 as a hosted page with no payment fields. We then had a production AI agent complete it: Claude, using the computer-use tools of the Claude desktop app's built-in browser, a headed Chromium. We recorded 6 sessions in two strategies. In *click mode* (4 sessions), the agent clicks each field at screen coordinates and types. In *keyboard mode* (2 sessions), it clicks once and moves between fields with Tab. The page stored only event counts and timings.

Table 6 shows what a risk SDK sees. Three things stand out. First, the agent *clicks without moving the mouse*: 7 clicks and zero mouse-move events, something no physical mouse produces. Second, typed text arrives as input events with no keydowns, as with the `insertText` primitive. In keyboard mode, the only keydowns are navigation keys, about 4.7 ms apart. Third, and contrary to our simulator's assumption, the session shows *no* webdriver flag and no headless user agent. A production agent running in a real desktop

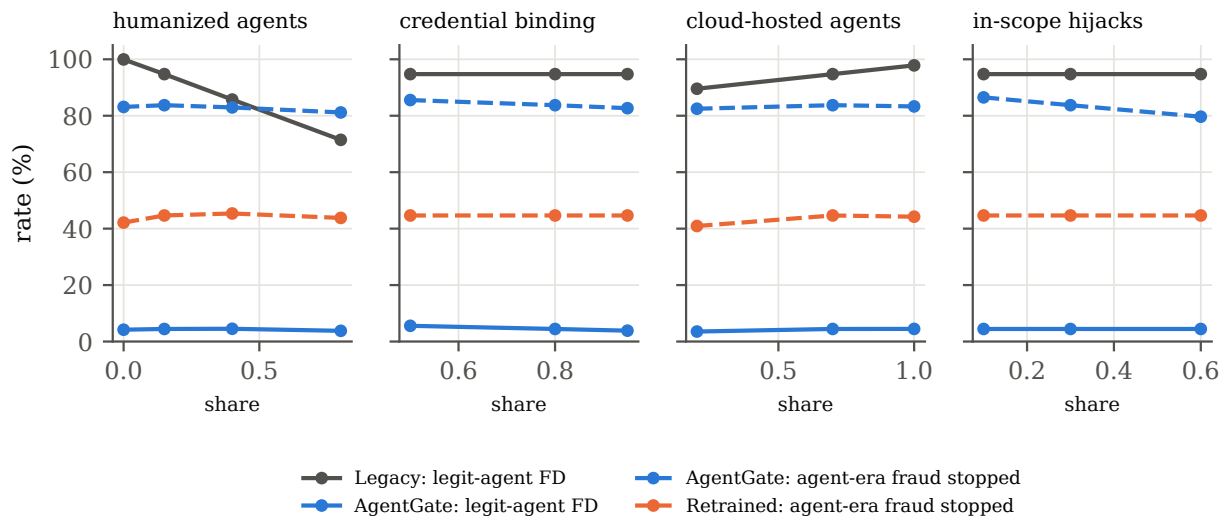


Figure 6: Sensitivity (3 seeds per point). Solid: false declines on legitimate agents. Dashed: mean catch rate on agent-era attacks.

Table 6: Client-side telemetry from a production computer-use agent on the hosted test checkout (means; time to submit is the median). No session produced a single mouse-move event.

Agent	mouse-moves	mouse-downs	keydown /input	untrusted events	submit (s)	webdriver flag	headless UA
Claude computer use, click mode (n=4)	0	7	0/6	0	15.3	0	0
Claude computer use, keyboard mode (n=2)	0	1	8/6	0	7.1	0	0

browser looks ordinary at the device level. Only its behavior gives it away.

We then attached each real behavioral vector to 2,000 draws of legitimate transaction context and scored it with the benchmark's models (3 seeds). Table 7 reports four deployment contexts: agent run on the user's machine (residential IP) or in the cloud (datacenter IP), and unverified or verified (signature plus mandate).

Finding 7 (real agent). For a real production agent, the conflation depends on where the agent runs. Run from the user's own machine, it is declined 18.9% of the time by the legacy model, about 4× the rate for human behavior in the same contexts (4.3%), but far below the simulated 94.5%. Run from the cloud, it is declined 88.5% of the time. Given a verified identity and mandate, AGENTGATE declines 1.9% (local) and 1.9% (cloud) of the same sessions.

This refines the headline result in two ways. The simulator overstated device-level evidence: real agents need not expose automation flags, so a legacy model leans on network context and on whatever behavioral features it has. Our feature set has no "click without movement" signal. A production risk SDK that tracks pointer paths would likely separate this agent from humans much more sharply, which would push local-agent false declines toward the cloud figures. On the other hand, the fix carries over: once delegation is verifiable, AGENTGATE treats the real agent like a customer, whatever the context. Without verification, real agents remain expensive under every policy (AGENTGATE declines 39.0% of unverified cloud sessions, versus 10.0% for human behavior in the same contexts). Signing is the agent developer's main lever. This is a small sample from one agent with two strategies, and we treat it as a check on the benchmark's assumptions rather than a population estimate.

Table 7: Share of the 6 real production-agent sessions declined (%) at a 2% human decline rate. Gray: the same contexts with simulated human behavior, for reference.

	Unverified agent				Verified agent			
	local		cloud		local		cloud	
Policy	agent	human	agent	human	agent	human	agent	human
Legacy	18.9	4.3	88.5	47.0	18.9	4.3	88.5	47.0
Retrained	8.6	4.4	30.0	9.9	8.6	4.4	30.0	9.9
AgentGate	10.7	4.3	39.0	10.0	1.9	0.1	1.9	0.0

7 Implications

For protocol designers. Identity is necessary but not sufficient. Our ablations show that the protective value lies in *delegation* (binding the credential to the principal who signed the mandate) and *scope* (machine-checkable constraints on merchant, category, amount and delivery). Protocols should (1) make the mandate, or a privacy-preserving commitment to its scope, available to the merchant's risk engine at authorization time; (2) include the delivery address and merchant identity in scope, applying least privilege [19] to spending authority, because hijacks most often redirect delivery; and (3) give the principal a standard step-up channel. AP2's cart mandates and ACP's merchant- and amount-scoped tokens already point this way [11, 21]. Our results give a quantitative reason to make them mandatory rather than optional.

For issuers and networks. Credential binding was the single largest lever on laundering. Binding lets an issuer check that the principal who delegated is the cardholder, which no merchant-side signal can establish.

For merchants and risk vendors. (1) Do not simply retrain on agent traffic or allowlist verified agents. Both create attack surfaces that standard metrics do not reveal. (2) Monitor agent false-decline rate as its own KPI, since aggregate approval rates hide it. (3) Route unattested automation to step-up authentication, not to decline. (4) Treat agent-platform onboarding quality (the KYC tier) as a risk input, in the same way BIN or IP reputation is used today.

8 Related Work

Agent fingerprinting. Recent measurement studies show that AI browsing agents can be told apart from humans and from each other using behavioral, browser and network fingerprints [9, 15, 25]. Wang et al. find that Cloudflare's bot detection caught one of seven agents while their classifier caught all seven [25]. Kang et al. report 97% accuracy across six frameworks [15]. Our work starts where these studies stop: given that agents are detectable, what should a payment decision do with that fact?

Agentic commerce security. Srivastava and Paul benchmark fraud *against* agents that spend money, such as overcharging counterparties [20]. We study the opposite direction, where agents are the actors being judged by merchant fraud models. TIVA proposes blockchain-based verification of agent identity and intent but does not evaluate it against fraud models or false declines [1]. Industry protocols define the building blocks [11, 17, 18, 21, 24]. We supply the first measurement of what happens when a merchant risk engine uses (or ignores) them.

Prompt injection in shopping agents. Indirect prompt injection [12] and its benchmarks [7, 26] show that shopping agents can be steered by page content. Our hijack class models the *payment-side* consequence. It also shows that defenses at the merchant can catch what agent-side defenses miss, provided the mandate is visible.

Fraud detection and bot detection. Card fraud detection research has studied realistic evaluation, class imbalance, feature engineering and drift [5, 6, 10], and bot detection has used mouse dynamics and fingerprint consistency [2, 14, 23]. Both lines of work treat automation as a negative signal. To our knowledge, AGENTRISK is the first benchmark in which automation is often legitimate. Web agent environments such as WebShop and WebArena [27, 28] evaluate task success rather than how risk systems treat agents.

9 Limitations and Ethics

Semi-synthetic data. Agent behavioral telemetry is grounded in real automation primitives, but human, fraud, transaction and delegation features are parametric. Absolute rates should be read as properties of the benchmark, not of any production system. The *qualitative* findings (the conflation, safe harbor, laundering, the verification paradox, and the value of scope and delegation checks) follow from structural facts: legitimate agents and bots share primitives, and identity does not imply authorization. They persist across every sweep we ran. We expose every parameter so that practitioners with production data can re-instantiate the benchmark. **Primitives and one production agent.** The benchmark is built from automation primitives measured in headless Chromium. Section 6.7 checks it against 6 sessions from one production agent in a headed browser, which is enough to correct one assumption (device-level flags) but not to estimate population rates. Larger studies across agent products, browsers and hosting setups are the most important next step. **Protocol assumptions.** We assume mandates are visible to the risk engine. Today this is true in some deployments and not others, and our results argue for making it standard. **Ethics.** We study defenses. The attack classes are already discussed publicly, and we release no attack tooling beyond session descriptors. The telemetry experiment used only a local test page and no third-party sites.

10 Conclusion

Fraud models learned a shortcut, "automated means risky," that the arrival of AI shopping agents has invalidated. Correcting it with more data or blanket trust creates new holes, because the shortcut stood in for a question that behavior cannot answer: was this purchase authorized by the person who pays? The answer has to come from verification of identity, delegation and scope, with learning used for whatever remains. AGENTRISK makes that argument testable, and we hope it becomes a shared yardstick while agentic commerce protocols are still being settled.

Availability. The simulator, telemetry harness, test checkout page, real-agent sessions, evaluation code and all result files are available at `https://github.com/RishabhPande/agentrisk`. Every figure and table in this paper is produced by scripts in that repository, and all numbers in the text are generated from result files rather than typed by hand.

Use of AI tools. The author used an AI assistant (Claude, by Anthropic) to help write code, run experiments and draft text. The author reviewed the work and takes full responsibility for the content.

References

[1] V. Acharya. Secure autonomous agent payments: Verifying authenticity and intent in a trustless environment. *arXiv preprint arXiv:2511.15712*, 2025.

[2] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez. BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection. *Pattern Recognition*, 127, 2022.

[3] A. Backman, J. Richer, and M. Sporny. HTTP message signatures. RFC 9421, 2024.

[4] Cloudflare. Securing agentic commerce: Helping AI agents transact with Visa and Mastercard. Cloudflare blog, Oct. 2025. URL `https://blog.cloudflare.com/secure-agentic-commerce/`.

[5] A. Correa Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten. Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51:134–142, 2016.

[6] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8):3784–3797, 2018.

[7] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

[8] C. Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.

[9] I. Fayolle, S. Bouhenniche, S. Pélissier, P. Laperdrix, C. Maurice, and W. Rudametkin. On the internet, nobody knows you're an LLM bot: Unmasking web agents with multi-layer fingerprinting. *arXiv preprint arXiv:2606.30119*, 2026.

[10] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 2014.

[11] Google. Agent payments protocol (AP2). Protocol specification, announced September 2025; donated to the FIDO Alliance in 2026, 2025. Summary: `https://www.spark.money/research/google-ap2-agent-payment-protocol`.

[12] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[13] N. Hardy. The confused deputy (or why capabilities might have been invented). In *ACM SIGOPS Operating Systems Review*, volume 22, pages 36–38, 1988.

[14] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris. Detection of advanced web bots by combining web logs with mouse behavioural biometrics. volume 2, 2021.

[15] D. Kang, H. Jeong, J. Sheffey, P. Datta, and A. Houmansadr. Whose agent are you? multi-layer fingerprinting and attribution of autonomous web agents. *arXiv preprint arXiv:2606.20910*, 2026.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.

[17] Mastercard. Mastercard unveils agent pay, pioneering agentic payments technology to power commerce in the age of AI. Press release, Apr. 2025. URL https://newsroom.mastercard.com/news/press/2025/april/mastercard-unveils-agent-pay-pioneering-agentic-payments-technology-to-power-commerce-in-the-age-of-ai.

[18] T. Meunier and S. Major. HTTP message signatures for bots. IETF Internet-Draft draft-ietf-webbotauth-httpsig-protocol-00, 2026. URL https://datatracker.ietf.org/doc/html/draft-ietf-webbotauth-httpsig-protocol-00.

[19] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[20] A. Srivastava and D. Paul. Agentic commerce bench: Measuring fraud detection for agents that spend money. *arXiv preprint arXiv:2609.35886*, 2026.

[21] Stripe. Stripe powers instant checkout in ChatGPT and releases agentic commerce protocol codeveloped with OpenAI. Press release, Sept. 2025. URL https://stripe.com/newsroom/news/stripe-openai-instant-checkout.

[22] The Fintech Times. AI shopping agents trigger 'false decline' crisis for merchants, warns Chargebacks911, May 2026. URL https://thefintechtimes.com/ai-shopping-agents-trigger-false-decline-crisis-for-merchants-warns-chargebacks911/.

[23] A. Vastel, P. Laperdrix, W. Rudametkin, and R. Rouvoy. FP-Scanner: The privacy implications of browser fingerprint inconsistencies. In *27th USENIX Security Symposium*, 2018.

[24] Visa. Visa introduces trusted agent protocol: An ecosystem-led framework for AI commerce. Press release, Oct. 2025. URL https://usa.visa.com/about-visa/newsroom/press-releases.releaseId.21716.html.

[25] E. Wang, Z. Shafiq, and Y. Vekaria. FP-Agent: Fingerprinting AI browsing agents. *arXiv preprint arXiv:2605.01247*, 2026.

[26] Z. Wang, Y. Li, Y. Wu, K. Chen, Z. Liu, F. K. Wai, P.-Y. Chen, V. L. L. Thing, B. Li, D. Tao, and T. Zhang. Who pays the price? stakeholder-centric prompt injection benchmarking for real-world web agents. *arXiv preprint arXiv:2606.13385*, 2026.

[27] S. Yao, H. Chen, J. Yang, and K. Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, 2022.

[28] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

A Simulator Parameters

Table 8: Key generator assumptions (defaults). All are parameters of `sim.py`.

Parameter	Default
Human device mix	40% mobile (no mouse events); 35% of desktop sessions use autofill (key-down/input $\sim U[0, 0.35]$)
Human typing	IKI log-normal, median 190 ms; time to submit median 95 s (35 s with autofill)
Human egress	4% datacenter/VPN
Agent primitive mix	DOM fill 40%, CDP insertText 20%, keystrokes 10%, computer-use 30%; 15% humanized
Agent hosting	70% cloud (shared egress, per-IP sessions median 800)
Agent planning latency	log-normal, median 25 s, added to primitive time
Agent checkout	45% guest; retries raise attempts/hour
Verified agents	platform KYC tier 0/1/2 = 5/25/70%; credential bound to principal 80%; 60% human-present mandates; 3% benign over-cap (tax/shipping)
Laundering (FAV)	platform tier 0/1/2 = 60/30/10%; principal account age $<$5 days; credential bound 5%
Hijack (FHJ)	30% stay in scope; others violate merchant/category/amount (40/30/30%); 30% of mandates merchant-agnostic
Classic fraud (FH)	40% new accounts; mismatch rates 2.5–3.5$\times$ legitimate; 35% paste card number
Bot fraud (FB)	35% card testing ($0.5–5, high velocity); 75% datacenter egress
Step-up	legitimate pass 92%; hijack victim approves 10%; fraudster passes 5%